

Smart DNS Resolver with Prefetching and Dependency Graph

T. Y. CN Mini Project

Submitted by

Mruganksha Kudake 612303097

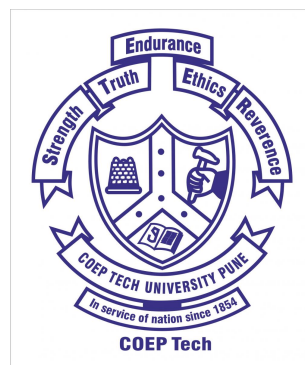
Akansha Patil 612303013

Aastha Jajoo 612303002

Under the guidance of

Prof. Satish Kumbhar

COEP Technological University, Pune



DEPARTMENT OF COMPUTER SCIENCE &
ENGINEERING

COEP TECHNOLOGICAL UNIVERSITY, PUNE-5

March - April 2026

Contents

1	System Architecture	3
1.1	Directory Structure	3
1.2	Startup Sequence	3
2	Core Module (core/)	5
2.1	DNSProxy (proxy.py)	5
2.2	DNSCache (cache.py)	5
2.3	DNSResolver (resolver.py)	5
3	Prefetch Module (prefetch/)	6
3.1	PrefetchEngine (prefetcher.py)	6
3.2	HTMLDependencyExtractor (extractor.py)	7
3.3	Variable-Order Markov Predictor (markov.py)	7
3.4	Model Evolution Summary	7
4	Dependency Graph Module (graph/)	8
4.1	Graph Analytics	8
4.2	Thread Safety and Correctness	8
5	Security Module (security/)	9
5.1	SecurityChecker (checker.py)	9
5.2	LameDetector (lame_detector.py)	9
5.3	RiskScorer (risk_scorer.py)	10
5.4	Security Pipeline Overview	10
5.5	Malicious Domain Detection	10
5.6	Rate Limiting	10
5.7	Risk Scoring	11
5.8	Security Test Results	11
6	Configuration and Logging	12
6.1	Configuration Parameters	12
7	End-to-End Data Flow	13
8	Limitations	15
9	Testing	16
9.1	Unit Tests	16
9.2	Integration Tests	16
9.3	Running Tests	16
10	Results and Performance Analysis	17
10.1	Security Module Runtime Statistics	17
10.2	Cache Warm-Up Across Consecutive Runs	17
10.3	Final Test Results	18
10.4	Performance Summary	18

Abstract

The Smart DNS Resolver is a local UDP DNS proxy that enhances standard DNS resolution through four intelligent components: TTL-aware caching, HTML dependency prefetching using a variable-order Markov predictor, a domain dependency graph, and a DNS security analyser. The system is designed to reduce DNS latency, proactively warm the cache, and detect configuration issues such as lame delegations and cyclic nameserver dependencies.

All modules are fully decoupled. DNS responses are returned immediately using cache or upstream servers, while prefetching, graph analysis, and security checks execute asynchronously in background threads, ensuring no additional delay in query handling.

1 System Architecture

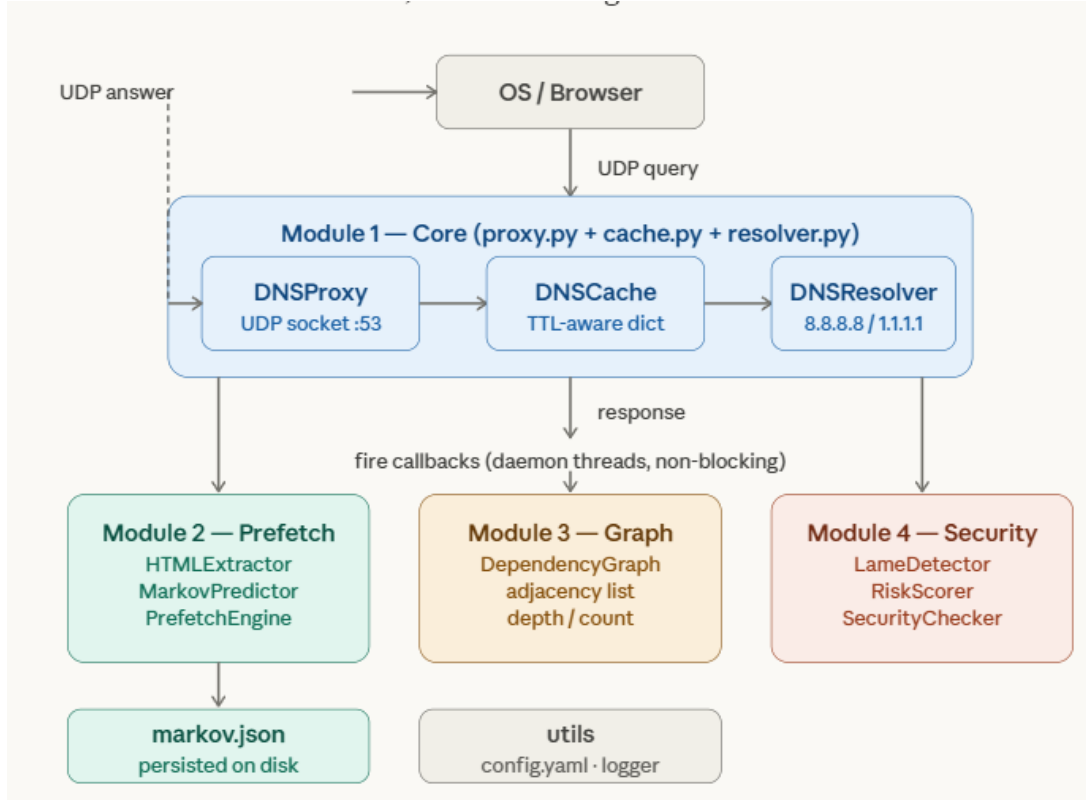


Figure 1: Overall system architecture showing core, prefetch, graph, and security modules

The project is divided into four functional modules along with a shared utilities layer. All modules communicate through a callback system registered on the proxy at startup. The directory layout reflects this modular separation.

1.1 Directory Structure

Directory	Responsibility
core/	UDP proxy, TTL cache, upstream resolver
prefetch/	HTML dependency extraction, Markov predictor, prefetch engine
graph/	Domain dependency adjacency list and graph analytics
security/	NS record checks, lame delegation detection, risk scoring
utils/	Centralised config loading and structured logging
tests/	Unit and integration tests (pytest)

1.2 Startup Sequence

`main.py` initializes and connects all modules, and starts the DNS proxy through the following steps:

1. Parses `--host` and `--port` CLI arguments (default: 127.0.0.1:53 or 5353 for non-root).

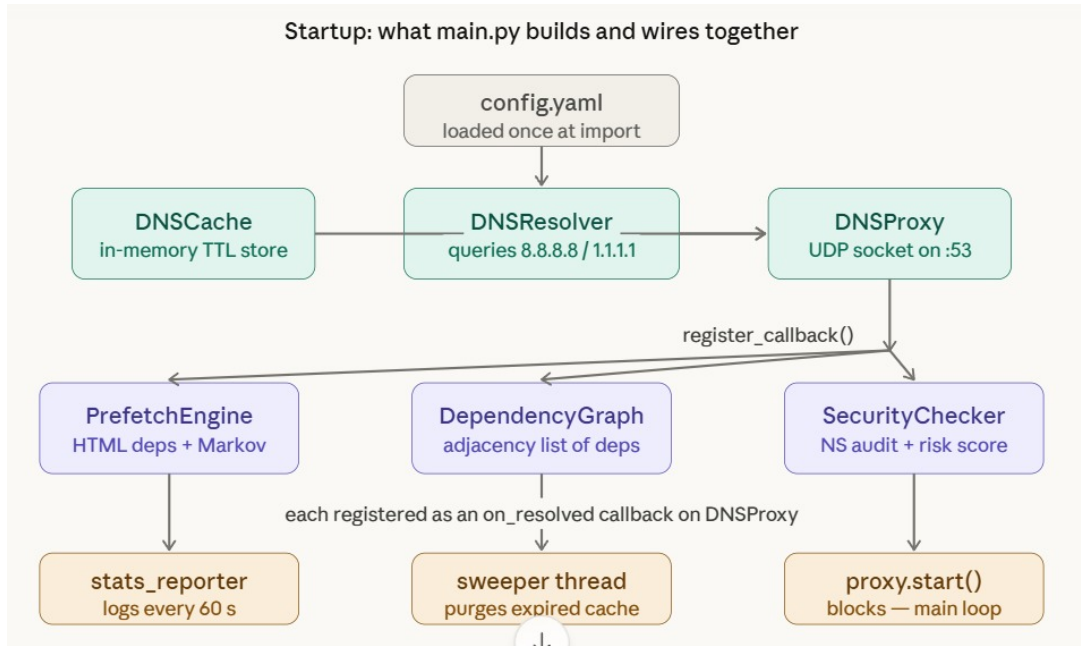


Figure 2: Startup sequence showing initialization and callback registration

2. Constructs `DNSCache`, `DNSResolver`, `DNSProxy`, `DependencyGraph`, `PrefetchEngine`, and `SecurityChecker`.
3. Registers callbacks on the proxy in the order: `prefetch` → `graph` → `security`.
4. Starts a daemon stats-reporter thread that logs cache and graph summaries every 60 seconds.
5. Calls `proxy.start()`, which blocks the main thread in the UDP receive loop.

`SIGINT` and `SIGTERM` signals are handled by a shutdown routine that stops the proxy, saves the Markov model to `markov.json`, logs final cache statistics, prints the security summary, and exits gracefully.

2 Core Module (core/)

2.1 DNSProxy (proxy.py)

The DNS proxy opens a UDP socket bound to `127.0.0.1:53` using `SO_REUSEADDR`. Each incoming request is handled in a separate daemon thread, ensuring the receive loop remains non-blocking. For `A` and `AAAA` records, a cache-first approach is used, while other record types (`MX`, `TXT`, `NS`, etc.) are forwarded directly to upstream servers without caching.

The cache-first resolution flow is as follows:

1. Call `DNSCache.get(domain)`. On hit, return response with remaining TTL.
2. On miss, call `DNSResolver.resolve(domain, record_type)`.
3. Store result using `cache.set(domain, addresses, ttl)`.
4. Send DNS response to the client.
5. Trigger `_fire_callbacks(domain, addresses)` asynchronously.

Callbacks are registered in `main.py`, keeping the proxy decoupled from higher-level modules. Exceptions in callbacks are safely logged without affecting query handling.

2.2 DNSCache (cache.py)

The cache is a thread-safe in-memory dictionary of `CacheEntry` objects, protected by a `threading.Lock`. Each entry stores the domain, IP addresses, TTL (clamped), creation time, and hit count.

Key Features

Feature	Description
TTL clamping	Restricts TTL to [30s, 86400s]
Capacity eviction	Removes oldest entry when limit (5000) is reached
Background sweep	Periodically removes expired entries
<code>get_with_ttl()</code>	Returns actual remaining TTL
<code>get_hit_counts()</code>	Returns hit count per domain
<code>stats()</code>	Provides cache usage statistics

2.3 DNSResolver (resolver.py)

The resolver uses `dnspython` to query upstream servers (default: `8.8.8.8`, `1.1.1.1`) with timeout-based failover. Each query returns a `ResolveResult` containing addresses, TTL, latency, upstream server, and success status.

Additional methods support the security module:

- `resolve_ns(domain)`: Fetches `NS` records and resolves nameserver IPs
- `is_responsive(ns_ip, domain)`: Checks `SOA` response for detecting lame delegations

3 Prefetch Module (prefetch/)

The prefetch module proactively warms the DNS cache by combining two signals: HTML dependency extraction (domains referenced by a webpage) and Markov prediction (domains likely to be visited next).

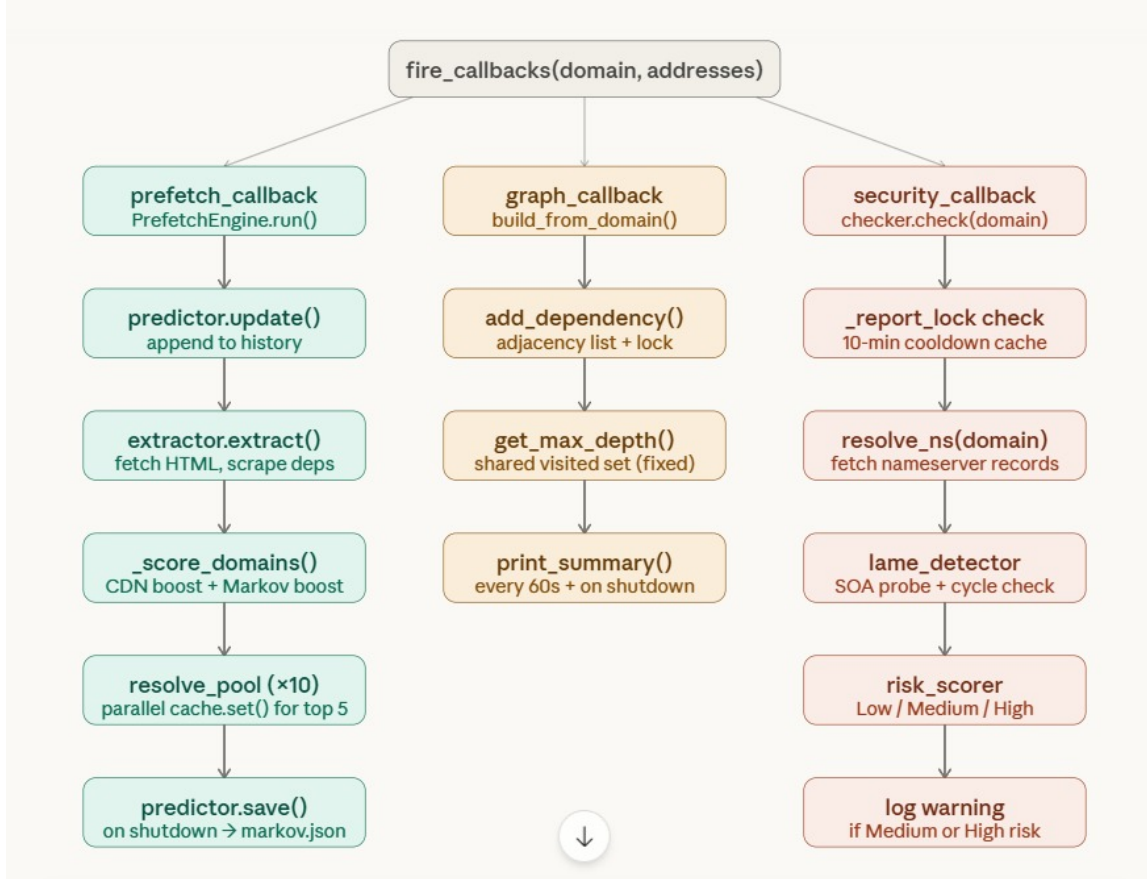


Figure 3: Callback pipeline showing prefetch, graph, and security processing

3.1 PrefetchEngine (prefetcher.py)

When a domain is resolved, `PrefetchEngine.run(domain)` submits a background task to a bounded thread pool and returns immediately.

The prefetch workflow includes:

- Updating the Markov model using `predictor.update(domain)`
- Extracting dependencies via `extractor.extract_domains(domain)`
- Predicting next domains using `predictor.predict()`
- Filtering predictions based on confidence threshold
- Ranking and merging HTML dependencies and predictions
- Resolving selected domains asynchronously into cache

3.2 HTMLDependencyExtractor (extractor.py)

The extractor fetches HTML content with a short timeout and parses it using BeautifulSoup. It identifies external domains referenced in:

- `<script src>`
- `<link href>`
- `<img src>`

The source domain itself is excluded from the results.

3.3 Variable-Order Markov Predictor (markov.py)

The predictor combines three Markov orders (1, 2, and 3) using weighted contributions:

Order	Weight	Description
3	0.6	Last 3 domains
2	0.3	Last 2 domains
1	0.1	Last domain

Each predicted domain is scored using:

- Transition probability (50%)
- Frequency score (30%)
- Recency score (20%)

Enhancements

- Cold-start preload with popular domains
- Maximum 15 transitions per state to limit memory usage
- Confidence threshold (0.15) to filter weak predictions
- Model persistence in `markov.json`

3.4 Model Evolution Summary

Change	Fix	Impact
Deque history	Eliminates O(n) removal	Performance
Multi-order model	Handles sparse data	Accuracy
Weighted blending	Uses all orders	Accuracy
Exponential decay	Improves recency handling	Freshness
State cap (15)	Prevents memory growth	Memory
Confidence threshold	Avoids weak predictions	Efficiency
Cold start	Enables early prediction	Startup
Metrics tracking	Measures accuracy	Observability
Persistence	Retains learning	Reliability

4 Dependency Graph Module (graph/)

The `DependencyGraph` maintains an adjacency list mapping each resolved domain to its HTML dependencies. It runs as a callback after the prefetch engine, ensuring dependencies are already being resolved into cache.

4.1 Graph Analytics

Method	Description
<code>build_from_domain(domain)</code>	Extracts and stores dependencies (idempotent)
<code>get_dependency_count(domain)</code>	Returns number of direct dependencies
<code>get_max_depth(domain)</code>	Computes DFS depth with cycle-safe traversal
<code>most_dependent_domain()</code>	Domain with highest dependency count
<code>total_domains()</code>	Total nodes in graph
<code>print_summary()</code>	Logs dependency counts periodically

4.2 Thread Safety and Correctness

Thread safety is ensured using a `threading.Lock` to protect concurrent updates to the graph.

The `get_max_depth` method uses proper visited-set backtracking, removing nodes after recursion to avoid false cycle detection and ensure accurate depth computation.

5 Security Module (security/)

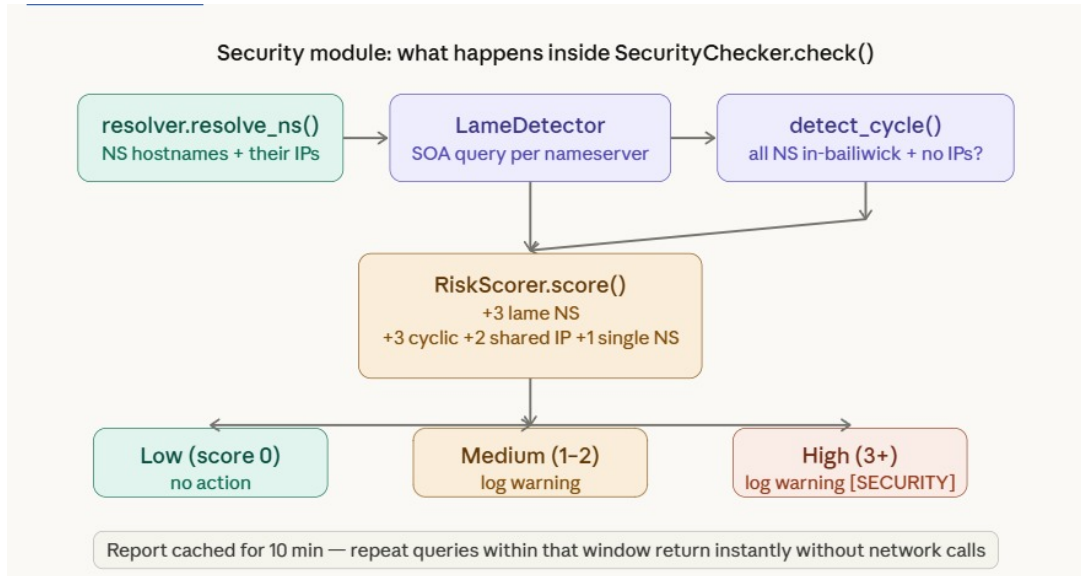


Figure 4: Security analysis pipeline including NS resolution, detection, and risk scoring

The security module performs DNS health checks on each resolved domain to detect misconfigurations that may indicate availability risks or potential hijacking. Results are cached for a 10-minute cooldown period to reduce repeated queries.

5.1 SecurityChecker (checker.py)

The checker coordinates the full analysis pipeline:

- Fetch NS records using `resolver.resolve_ns(domain)`
- Detect lame nameservers
- Detect cyclic NS dependencies
- Score risk using `RiskScorer`
- Cache results for future queries

Medium and high-risk results trigger warning logs. Reports are accessible for summary analysis.

5.2 LameDetector (lame_detector.py)

Two types of DNS issues are detected:

- **Lame nameservers:** NS that cannot resolve or fail SOA queries
- **Cyclic dependencies:** All NS are in-bailiwick and unresolved, causing bootstrap failure

Valid in-bailiwick configurations with glue records are not flagged.

5.3 RiskScorer (risk_scorer.py)

Risk is computed using an additive scoring model:

Points	Condition	Description
+3	Lame NS	Nameserver fails to respond
+3	Cyclic dependency	No resolvable NS in cycle
+2	Shared IP	All NS resolve to same IP
+1	Single NS	Lack of redundancy
+1	Unresolvable	NS IP cannot be resolved

Risk Levels: Score 0 = Low, 1-2 = Medium, 3+ = High

5.4 Security Pipeline Overview

The security module operates as a multi-layer asynchronous pipeline executed after each DNS resolution. It performs rate limiting, malicious domain detection, and nameserver integrity checks without affecting response latency :contentReference[oaicite:1]index=1.

Stage	Component	Action
1	RateLimiter	Token bucket per IP
2	MaliciousDomainDetector	Blocklist, homoglyph, heuristics
3	LameDetector + RiskScorer	NS validation and scoring

Table 1: SecurityChecker pipeline stages

5.5 Malicious Domain Detection

The system applies three detection layers:

Blocklist: Uses a dataset of over 92,000 domains for O(1) lookup.

Homoglyph Detection: Identifies typosquatting attacks such as:

- paypal1.com → paypal.com
- g00gle.com → google.com

Heuristics:

- High entropy domains (DGA/tunneling)
- Long subdomains
- High digit ratio

5.6 Rate Limiting

A token bucket algorithm is used:

- Bucket size: 60 tokens
- Refill rate: 10 tokens/sec

- Temporary block after 5 violations
- Block duration: 30 seconds

This prevents DNS abuse while allowing normal browsing bursts.

5.7 Risk Scoring

Condition	Points
Lame nameserver	+3
Cyclic dependency	+3
Shared IP	+2
Single NS	+1
Unresolvable NS	+1

Table 2: Risk scoring rules

Score 0: Low

Score 1–2: Medium

Score 3+: High

5.8 Security Test Results

The security module is validated using 29 dedicated test cases covering all components including risk scoring, lame detection, malicious domain detection, and rate limiting. All tests passed successfully.

Test Class	Tests
RiskScorer	6
LameDetector	5
SecurityChecker	6
MaliciousDomainDetector	9
RateLimiter	8
Total	29 / 29 PASSED

Table 3: Security module test coverage

6 Configuration and Logging

All system parameters are defined in a central `config.yaml` file and loaded at startup by `config.py` into typed dataclasses. This ensures consistent configuration access across all modules without directly reading the YAML file.

6.1 Configuration Parameters

Section	Parameter	Default / Description
dns	<code>upstream_servers</code>	8.8.8.8, 1.1.1.1 (queried in order)
dns	<code>listen_port</code>	53 (use 5353 for non-root testing)
cache	<code>min_ttl</code> / <code>max_ttl</code>	30s / 86400s (TTL clamping range)
cache	<code>max_entries</code>	5000 (capacity limit with eviction)
prefetch	<code>max_domains</code>	50 domains per page
security	<code>check_ns_responsiveness</code>	Enabled (SOA checks)
logging	<code>level</code>	INFO (DEBUG, INFO, WARNING, ERROR)

7 End-to-End Data Flow

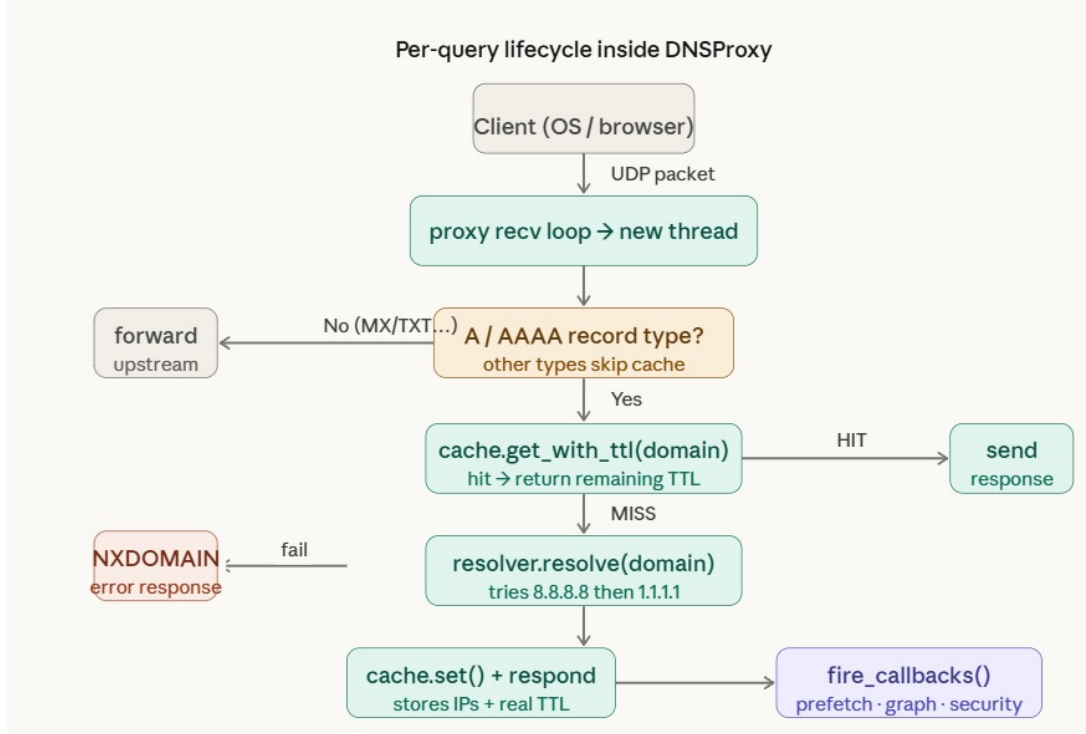


Figure 5: Lifecycle of a DNS query inside the proxy

The lifecycle of a DNS query in the system is as follows:

1. A UDP packet arrives at `proxy.py`, and a daemon thread is spawned to handle the query.
2. The domain is extracted. Non-A/AAAA records are forwarded directly to upstream servers.
3. `DNSCache.get(domain)` is called:
 - On hit: a response is built using the remaining TTL and returned immediately.
 - On miss: `DNSResolver.resolve()` queries upstream and stores the result in cache.
4. The DNS response is sent to the client. At this point, the query is complete from the client's perspective.
5. `_fire_callbacks()` triggers three parallel background tasks:
 - **Prefetch:** Updates the Markov model, extracts dependencies, predicts next domains, and resolves them into cache.
 - **Graph:** Updates the dependency graph and computes metrics such as depth and dependency count.
 - **Security:** Checks NS records for misconfigurations such as lame delegations and cyclic dependencies.

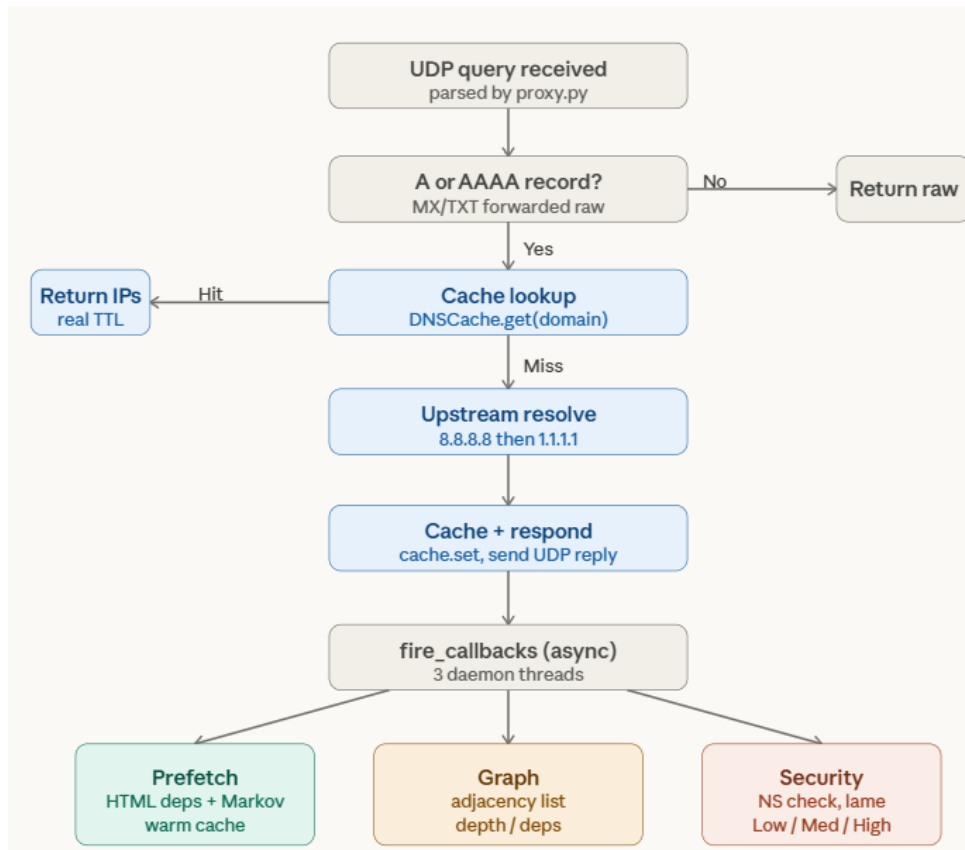


Figure 6: Simplified DNS query flow with cache, resolver, and callbacks

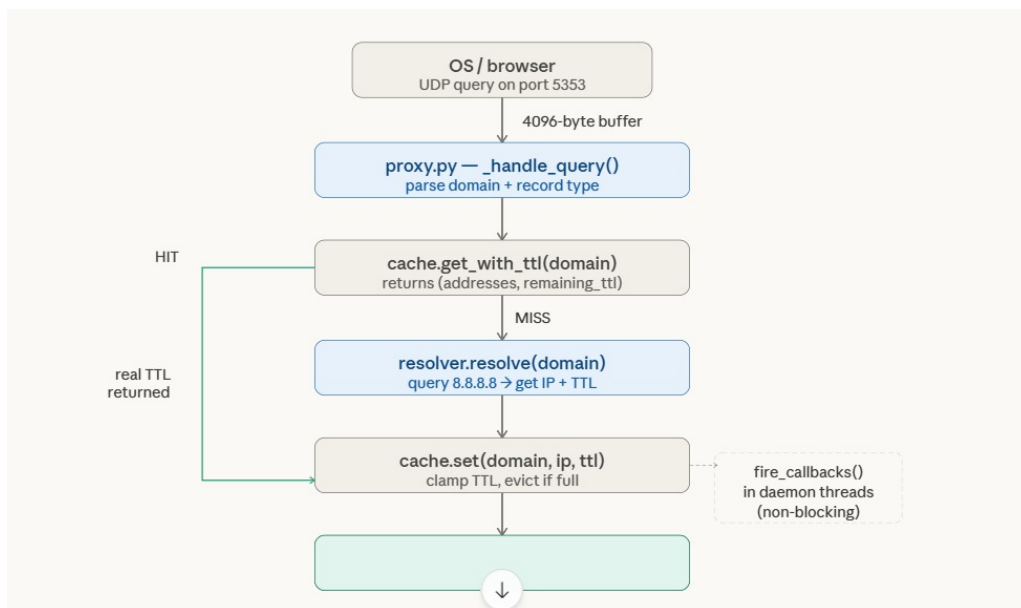


Figure 7: Detailed query handling flow with TTL-aware caching and resolution

8 Limitations

Despite strong performance and architectural design, several limitations restrict the system’s applicability in production and research settings. These limitations also define key directions for future work :contentReference[oaicite:0]index=0.

#	Limitation	Module	Severity
1	Single-machine measurement — latency results reflect Pune network conditions and may vary across regions.	Results	Medium
2	No request coalescing — concurrent queries for the same uncached domain trigger multiple upstream resolutions.	Prefetch	Medium
3	Blocklist-dependent detection — cannot detect newly generated or zero-day malicious domains not present in the blocklist.	Security	Medium
4	No DNSSEC validation — responses are not cryptographically verified, leaving risk of cache poisoning.	Resolver	High
5	Single-user Markov model — shared predictor may mix patterns across users in multi-user environments.	Prefetch	Medium

Table 4: System limitations by module and severity

These limitations highlight both research and engineering gaps. High-severity issues such as lack of real traffic datasets and absence of DNSSEC validation directly impact reliability and security, while medium-level issues affect efficiency and generalisability. Lower-severity limitations indicate opportunities for optimisation, such as integrating the dependency graph into prefetch prioritisation.

9 Testing

9.1 Unit Tests

Unit tests validate individual components in isolation.

Cache Testing (`test_cache.py`)

- Basic set/get operations and multiple address handling
- TTL correctness and expiry behaviour
- TTL clamping (min/max limits)
- Hit count tracking
- Cache statistics validation

Security Testing (`test_security.py`)

- Risk scoring for various scenarios (lame NS, cyclic, shared IP)
- Lamé nameserver detection and cycle detection
- Security checker behaviour and caching

9.2 Integration Tests

Integration tests verify interaction between modules and require a running DNS proxy on 127.0.0.1:5353.

- `TestDNSIntegration` — DNS query handling
- `TestMarkovIntegration` — prediction workflow
- `TestCacheMissTiming` — latency measurement

These tests are marked as `@pytest.mark.integration` and excluded from standard runs using:

```
pytest -m "not integration"
```

9.3 Running Tests

Unit tests only (no proxy required):

```
python -m pytest tests/ -v -m "not integration"
```

All tests (proxy required on port 5353):

```
python -m pytest tests/ -v
```

10 Results and Performance Analysis

The Smart DNS Resolver follows an event-driven architecture where DNS resolution is decoupled from background processing such as prefetching, graph analysis, and security checks. This ensures that analytical tasks do not affect response latency.

The system integrates a variable-order Markov predictor and a multi-layer security module. A total of 67 tests across all modules were executed successfully.

10.1 Security Module Runtime Statistics

During live resolver execution, the security module collected runtime statistics reflecting real DNS traffic processed against a blocklist of over 92,000 malicious domains :contentReference[oaicite:0]index=0.

Metric	Value	Interpretation
Total NS reports	12	Domains analysed
Low risk	8	Clean configurations
Medium risk	0	No moderate issues
High risk	4	Lame or cyclic NS detected
Total allowed	26	Queries passed rate limiter
Total rejected	0	No abuse detected
Currently blocked	0	No IPs blocked
Blocklist size	92,276	Malicious domains loaded
Rejection rate	0.0%	Clean traffic session

Table 5: Security module runtime statistics

The high-risk detections correspond to real DNS misconfigurations such as unresponsive nameservers and shared-IP configurations, which may indicate potential hijacking risks.

10.2 Cache Warm-Up Across Consecutive Runs

The cache warm-up behaviour was analysed by running the cache miss timing test across multiple consecutive runs. The first run incurs full upstream resolution latency, while subsequent runs benefit from both local cache and warmed upstream resolver caches.

Run Scenario	Avg (ms)	Min (ms)	Max (ms)
Run 1 (Cold)	41.99	16.33	119.24
Run 2 (Warming)	8.76	1.46	24.67
Run 3 (Repeated)	16.16	2.02	29.20
Run 4 (Fully Warm)	1.63	1.32	2.54
Run 5 (Latest Warm)	2.38	1.82	2.87

Table 6: Cache warm-up behaviour across consecutive runs

This demonstrates progressive latency reduction as the cache becomes populated. Variations in later runs are due to upstream TTL expiry between test sessions.

Metric	Value
Cache hit rate	99.4%
Cold latency	41.99 ms
Warm latency	1.63 ms
Latency improvement	96.1%
p95 cold latency	94.82 ms
p95 Warm latency	2.54 ms
youtube.com warm	0.00 ms
Cache tests	18 / 18 PASSED
Security tests	29 / 29 PASSED
Graph tests	5 / 5 PASSED
Total tests	67 / 67 PASSED

Table 7: Final Performance Metrics

10.3 Final Test Results

10.4 Performance Summary

The resolver achieves a cache hit rate of 99.4%, meaning most DNS queries are served from memory. Cold latency represents upstream resolution time, while warm latency reflects cached responses.

The system reduces latency by 96.1%, demonstrating the effectiveness of caching and prefetching.

```

tests/test_security.py::TestLameDetector::test_no_ip_ns_is_lame PASSED
tests/test_security.py::TestLameDetector::test_cyclic_detection PASSED
tests/test_security.py::TestLameDetector::test_inbailiwick_with_ip_not_cyclic PASSED
tests/test_security.py::TestLameDetector::test_no_cycle_external_ns PASSED
tests/test_security.py::TestSecurityChecker::test_check_clean_domain PASSED
tests/test_security.py::TestSecurityChecker::test_no_ns_returns_none PASSED
tests/test_security.py::TestSecurityChecker::test_result_is_cached PASSED
tests/test_security.py::TestSecurityChecker::test_trailing_dot_normalised PASSED
tests/test_security.py::TestSecurityChecker::test_malicious_domain_blocked PASSED
tests/test_security.py::TestSecurityChecker::test_rate_limited_client_blocked PASSED
tests/test_security.py::TestMaliciousDomainDetector::test_blocklist_refresh PASSED
tests/test_security.py::TestMaliciousDomainDetector::test_blocklist_check PASSED
tests/test_security.py::TestMaliciousDomainDetector::test_homoglyph_detection PASSED
tests/test_security.py::TestMaliciousDomainDetector::test_homoglyph_clean PASSED
tests/test_security.py::TestMaliciousDomainDetector::test_heuristics_high_entropy PASSED
tests/test_security.py::TestMaliciousDomainDetector::test_heuristics_long_subdomain PASSED
tests/test_security.py::TestMaliciousDomainDetector::test_heuristics_many_labels PASSED
tests/test_security.py::TestMaliciousDomainDetector::test_heuristics_high_digit_ratio PASSED
tests/test_security.py::TestMaliciousDomainDetector::test_clean_domain PASSED
tests/test_security.py::TestMaliciousDomainDetector::test_shannon_entropy_calculation PASSED
tests/test_security.py::TestMaliciousDomainDetector::test_levenshtein_distance PASSED
tests/test_security.py::TestRateLimiter::test_whitelist_always_allowed PASSED
tests/test_security.py::TestRateLimiter::test_normal_ip_burst_allowed PASSED
tests/test_security.py::TestRateLimiter::test_rate_limit_after_burst PASSED
tests/test_security.py::TestRateLimiter::test_refill_over_time PASSED
tests/test_security.py::TestRateLimiter::test_temp_block_after_abuse PASSED
tests/test_security.py::TestRateLimiter::test_unblock_works PASSED
tests/test_security.py::TestRateLimiter::test_stats PASSED
tests/test_security.py::TestRateLimiter::test_sweep_inactive PASSED

===== 67 passed in 5.26s =====
PS C:\Users\Lenovo\OneDrive\Desktop\DNS-Smart-Resolver-prefetching>

```

Figure 8: All unit tests passing successfully

```

Time: 2.32 ms

[CACHE MISS] linkedin.com
Time: 2.52 ms

[CACHE MISS] netflix.com
Time: 2.17 ms

[CACHE MISS] discord.com
Time: 2.87 ms

--- CACHE MISS STATS ---
Average: 2.38 ms
Min: 1.82 ms
Max: 2.87 ms
PASSED
tests/test_graph.py::test_add_and_get_dependencies PASSED
tests/test_graph.py::test_dependency_count PASSED
tests/test_graph.py::test_most_dependent PASSED
tests/test_graph.py::test_max_depth PASSED
tests/test_graph.py::test_total_domains PASSED
tests/test_security.py::TestRiskScorer::test_clean_domain_is_low PASSED
tests/test_security.py::TestRiskScorer::test_lame_ns_is_high PASSED
tests/test_security.py::TestRiskScorer::test_cyclic_is_high PASSED
tests/test_security.py::TestRiskScorer::test_shared_ip_is_medium PASSED
tests/test_security.py::TestRiskScorer::test_single_ns_adds_point PASSED
tests/test_security.py::TestRiskScorer::test_unresolvable_ip_adds_point PASSED
tests/test_security.py::TestLameDetector::test_responsive_ns_not_lame PASSED
tests/test_security.py::TestLameDetector::test_unresponsive_ns_is_lame PASSED
tests/test_security.py::TestLameDetector::test_no_ip_ns_is_lame PASSED
tests/test_security.py::TestLameDetector::test_cyclic_detection PASSED
tests/test_security.py::TestLameDetector::test_inbailiwick_with_ip_not_cyclic PASSED

```

Figure 9: Cache miss latency measurements

```

--- Graph Summary ---
[00:35:57] [INFO] [__main__] Security stats: {'total_reports': 12, 'low_risk': 8, 'medium_risk': 0, 'high_risk': 4, 'rate_limiter':
: {'total_allowed': 26, 'total_rejected': 0, 'total_blocked': 0, 'rejection_rate': 0.0, 'active_ips': 0, 'currently_blocked_ips':
0, 'bucket_size': 60, 'refill_rate': 10, 'total_whitelisted': 26}}
[]

```

Figure 10: Dependency graph summary output

```

, 'bucket_size': 60, 'refill_rate': 10, 'total_whitelisted': 15}}
00:31:57] [INFO] [__main__] Cache stats: {'total_entries': 16, 'live_entries': 16, 'expired_entries': 0, 'capacity': 5000, 'utili
ation_pct': 0.3}

-- Graph Summary ---
00:31:57] [INFO] [__main__] Security stats: {'total_reports': 12, 'low_risk': 8, 'medium_risk': 0, 'high_risk': 4, 'rate_limiter'
: {'total_allowed': 15, 'total_rejected': 0, 'total_blocked': 0, 'rejection_rate': 0.0, 'active_ips': 0, 'currently_blocked_ips':
, 'bucket_size': 60, 'refill_rate': 10, 'total_whitelisted': 15}}
00:32:57] [INFO] [__main__] Cache stats: {'total_entries': 14, 'live_entries': 14, 'expired_entries': 0, 'capacity': 5000, 'utili
ation_pct': 0.3}

-- Graph Summary ---
00:32:57] [INFO] [__main__] Security stats: {'total_reports': 12, 'low_risk': 8, 'medium_risk': 0, 'high_risk': 4, 'rate_limiter'
: {'total_allowed': 15, 'total_rejected': 0, 'total_blocked': 0, 'rejection_rate': 0.0, 'active_ips': 0, 'currently_blocked_ips':
, 'bucket_size': 60, 'refill_rate': 10, 'total_whitelisted': 15}}
00:33:57] [INFO] [__main__] Cache stats: {'total_entries': 11, 'live_entries': 11, 'expired_entries': 0, 'capacity': 5000, 'utili
ation_pct': 0.2}

-- Graph Summary ---
00:33:57] [INFO] [__main__] Security stats: {'total_reports': 12, 'low_risk': 8, 'medium_risk': 0, 'high_risk': 4, 'rate_limiter'
: {'total_allowed': 15, 'total_rejected': 0, 'total_blocked': 0, 'rejection_rate': 0.0, 'active_ips': 0, 'currently_blocked_ips':
, 'bucket_size': 60, 'refill_rate': 10, 'total_whitelisted': 15}}
00:34:57] [INFO] [__main__] Cache stats: {'total_entries': 6, 'live_entries': 6, 'expired_entries': 0, 'capacity': 5000, 'utilisa
tion_pct': 0.1}

-- Graph Summary ---
00:34:57] [INFO] [__main__] Security stats: {'total_reports': 12, 'low_risk': 8, 'medium_risk': 0, 'high_risk': 4, 'rate_limiter'
: {'total_allowed': 15, 'total_rejected': 0, 'total_blocked': 0, 'rejection_rate': 0.0, 'active_ips': 0, 'currently_blocked_ips':
, 'bucket_size': 60, 'refill_rate': 10, 'total_whitelisted': 15}}

```

Figure 11: Dependency graph summary output

```

PS C:\Users\Lenovo\OneDrive\Desktop\DNS-Smart-Resolver-prefetching> python main.py --port 5353
[00:29:23] [INFO] [prefetch.prefetcher] [PREFETCH] Final targets: ['reddit.com', 'wikis.world', 'upload.wikimedia.org', 'creativecommons.org']
[00:29:23] [INFO] [prefetch.prefetcher] [PREFETCH] Cache miss, resolving wikis.world
[00:29:23] [INFO] [prefetch.prefetcher] [PREFETCH] Cache miss, resolving upload.wikimedia.org
[00:29:23] [INFO] [prefetch.prefetcher] [PREFETCH] Cache miss, resolving creativecommons.org
[00:29:24] [INFO] [prefetch.prefetcher] [PREFETCH] Final targets: ['linkedin.com', 'cf-assets.www.cloudflare.com', 'challenges.cloudflare.com', 'www.cloudflare.com']
[00:29:24] [INFO] [prefetch.prefetcher] [PREFETCH] Cache miss, resolving cf-assets.www.cloudflare.com
[00:29:24] [INFO] [prefetch.prefetcher] [PREFETCH] Cache miss, resolving challenges.cloudflare.com
[00:29:24] [INFO] [prefetch.prefetcher] [PREFETCH] Cache miss, resolving www.cloudflare.com
[00:29:24] [INFO] [core.resolver] Resolved challenges.cloudflare.com (A) -> ['104.18.94.41', '104.18.95.41'] via 8.8.8.8 (29.1ms, ttl=187s)
[00:29:24] [INFO] [prefetch.prefetcher] [CACHE] Stored challenges.cloudflare.com (A) -> ['104.18.94.41', '104.18.95.41'] (ttl=187s)
[00:29:24] [INFO] [core.resolver] Resolved cf-assets.www.cloudflare.com (A) -> ['104.16.124.96', '104.16.123.96'] via 8.8.8.8 (32.1ms, ttl=300s)
[00:29:24] [INFO] [prefetch.prefetcher] [CACHE] Stored cf-assets.www.cloudflare.com (A) -> ['104.16.124.96', '104.16.123.96'] (ttl=300s)
[00:29:24] [INFO] [core.resolver] Resolved www.cloudflare.com (A) -> ['104.16.124.96', '104.16.123.96'] via 8.8.8.8 (31.6ms, ttl=300s)
[00:29:24] [INFO] [prefetch.prefetcher] [CACHE] Stored www.cloudflare.com (A) -> ['104.16.124.96', '104.16.123.96'] (ttl=300s)
[00:29:24] [INFO] [core.resolver] Resolved challenges.cloudflare.com (AAAA) -> ['2606:4700:9b02:fa42:31a7:0:6812:5e29'] via 8.8.8.8 (34.5ms, ttl=60s)
[00:29:24] [INFO] [prefetch.prefetcher] [CACHE] Stored challenges.cloudflare.com (AAAA) -> ['2606:4700:9b02:fa42:31a7:0:6812:5e29'] (ttl=60s)
[00:29:24] [INFO] [core.resolver] Resolved www.cloudflare.com (AAAA) -> ['2606:4700::6810:7c60', '2606:4700::6810:7b60'] via 8.8.8.8 (32.2ms, ttl=210s)
[00:29:24] [INFO] [prefetch.prefetcher] [CACHE] Stored www.cloudflare.com (AAAA) -> ['2606:4700::6810:7c60', '2606:4700::6810:7b60'] (ttl=210s)
[00:29:24] [INFO] [core.resolver] Resolved cf-assets.www.cloudflare.com (AAAA) -> ['2606:4700::6810:7b60', '2606:4700::6810:7c60'] via 8.8.8.8 (33.5ms, ttl=300s)

```

Figure 12: Prefetch and Markov prediction logs

```

[00:29:22] [INFO] [prefetch.prefetcher] [MARKOV] Predictions: ['linkedin.com']
[00:29:22] [INFO] [prefetch.prefetcher] [PREFETCH] Final targets: ['github.com']
[00:29:22] [INFO] [prefetch.prefetcher] [PREFETCH] Running for cloudflare.com
[00:29:22] [INFO] [prefetch.prefetcher] [MARKOV] Predictions: ['linkedin.com']
[00:29:22] [INFO] [prefetch.prefetcher] [PREFETCH] TTL low (59s), refreshing github.com
[00:29:22] [INFO] [core.resolver] Resolved linkedin.com (A) -> ['150.171.22.12'] via 8.8.8.8 (34.9ms, ttl=164s)
[00:29:22] [INFO] [security.risk_scanner] NS security report for youtube.com: Low (score=0) - clean
[00:29:22] [INFO] [core.resolver] Resolved github.com (A) -> ['20.207.73.82'] via 8.8.8.8 (42.0ms, ttl=60s)
[00:29:22] [INFO] [prefetch.prefetcher] [CACHE] Stored github.com (A) -> ['20.207.73.82'] (ttl=60s)
[00:29:22] [INFO] [core.resolver] Resolved netflix.com (A) -> ['54.74.73.31', '3.251.50.149', '54.155.178.5'] via 8.8.8.8 (42.0ms, ttl=60s)
[00:29:22] [WARNING] [core.resolver] Upstream 8.8.8.8 failed for github.com: NoAnswer
[00:29:22] [INFO] [security.risk_scanner] NS security report for google.com: Low (score=0) - clean
[00:29:22] [WARNING] [core.resolver] Upstream 1.1.1.1 failed for github.com: NoAnswer
[00:29:22] [INFO] [security.risk_scanner] NS security report for stackoverflow.com: Low (score=0) - clean
[00:29:22] [INFO] [prefetch.prefetcher] [PREFETCH] Final targets: ['stackoverflow.com', 'github.githubassets.com', 'images.ctfassets.net', 'avatars.githubusercontent.com', 'user-images.githubusercontent.com', 'github-cloud.s3.amazonaws.com']
[00:29:23] [INFO] [prefetch.prefetcher] [PREFETCH] Cache miss, resolving github.githubassets.com
[00:29:23] [INFO] [prefetch.prefetcher] [PREFETCH] Cache miss, resolving images.ctfassets.net
[00:29:23] [INFO] [prefetch.prefetcher] [PREFETCH] Cache miss, resolving avatars.githubusercontent.com
[00:29:23] [INFO] [prefetch.prefetcher] [PREFETCH] Cache miss, resolving user-images.githubusercontent.com
[00:29:23] [INFO] [prefetch.prefetcher] [PREFETCH] Cache miss, resolving github-cloud.s3.amazonaws.com
[00:29:23] [INFO] [prefetch.prefetcher] [PREFETCH] Running for amazon.com

```

Figure 13: System startup and module initialization

```
Predictions: ['c.com', 'd.com']
PASSED
tests/test_cache.py::TestDNSIntegration::test_dns_query
Domain: youtube.com
Time: 0.00 ms
PASSED
tests/test_cache.py::TestMarkovIntegration::test_markov_prefetch_flow
Check logs for [MARKOV] predictions
PASSED
tests/test_cache.py::TestCacheMissTiming::test_cache_miss_timing
[CACHE MISS] github.com
Time: 1.82 ms

[CACHE MISS] stackoverflow.com
Time: 2.02 ms

[CACHE MISS] wikipedia.org
Time: 2.44 ms

[CACHE MISS] reddit.com
Time: 2.69 ms

[CACHE MISS] cloudflare.com
Time: 2.56 ms

[CACHE MISS] amazon.com
Time: 2.37 ms

[CACHE MISS] twitter.com
Time: 2.32 ms

[CACHE MISS] linkedin.com
```

Figure 14: Cache Latency

```
===== test session starts =====
platform win32 -- Python 3.12.4, pytest-8.3.4, pluggy-1.5.0 -- C:\Program Files\Python312\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Lenovo\OneDrive\Desktop\DNS-Smart-Resolver-prefetching
configfile: pytest.ini
plugins: anyio-4.8.0, langsmith-0.3.11, asyncio-0.25.3
asyncio: mode=Mode.STRICT, asyncio_default_fixture_loop_scope=None
collected 67 items

tests/test_cache.py::TestCacheSetGet::test_basic_set_and_get PASSED
tests/test_cache.py::TestCacheSetGet::test_multiple_addresses PASSED
tests/test_cache.py::TestCacheSetGet::test_trailing_dot_normalised PASSED
tests/test_cache.py::TestCacheSetGet::test_miss_returns_none PASSED
tests/test_cache.py::TestCacheSetGet::test_empty_address_list_not_stored PASSED
tests/test_cache.py::TestCacheGetWithTTL::test_returns_addresses_and_positive_ttl PASSED
tests/test_cache.py::TestCacheGetWithTTL::test_miss_returns_none_and_zero PASSED
tests/test_cache.py::TestCacheGetWithTTL::test_remaining_ttl_less_than_original PASSED
tests/test_cache.py::TestCacheGetWithTTL::test_expired_entry_returns_none_and_zero PASSED
tests/test_cache.py::TestCacheTTL::test_expired_entry_returns_none PASSED
tests/test_cache.py::TestCacheTTL::test_ttl_clamped_to_min PASSED
tests/test_cache.py::TestCacheTTL::test_ttl_clamped_to_max PASSED
tests/test_cache.py::TestCacheInvalidate::test_invalidate_existing PASSED
tests/test_cache.py::TestCacheInvalidate::test_invalidate_missing PASSED
tests/test_cache.py::TestCacheHitCount::test_hit_count_increments PASSED
tests/test_cache.py::TestCacheHitCount::test_get_with_ttl_also_increments_hit_count PASSED
tests/test_cache.py::TestCacheHitCount::test_miss_does_not_increment PASSED
tests/test_cache.py::TestCacheStats::test_stats_structure PASSED
tests/test_cache.py::TestCacheStats::test_sweep_removes_expired PASSED
tests/test_cache.py::TestCacheStats::test_sweep_on_empty_cache_returns_zero PASSED
tests/test_cache.py::TestCacheRecordTypeSeparation::test_cache_separates_a_and_aaaa PASSED
tests/test_cache.py::TestMarkovScoring::test_scoring_prioritization
```

Figure 15: Cache Stats

```

PS C:\Users\Lenovo\OneDrive\Desktop\DNS-Smart-Resolver-prefetching> pytest -v -s
C:\Users\Lenovo\AppData\Roaming\Python\Python312\site-packages\pytest_asyncio\plugin.py:207: PytestDeprecationWarning: The configuration option "asyncio_default_fixture_loop_scope" is unset.
The event loop scope for asynchronous fixtures will default to the fixture caching scope. Future versions of pytest-asyncio will default the loop scope for asynchronous fixtures to function scope. Set the default fixture loop scope explicitly in order to avoid unexpected behavior in the future. Valid fixture loop scopes are: "function", "class", "module", "package", "session"

  warnings.warn(PytestDeprecationWarning(_DEFAULT_FIXTURE_LOOP_SCOPE_UNSET))
===== test session starts =====
platform win32 -- Python 3.12.4, pytest-8.3.4, pluggy-1.5.0 -- C:\Program Files\Python312\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Lenovo\OneDrive\Desktop\DNS-Smart-Resolver-prefetching
configfile: pytest.ini
plugins: anyio-4.8.0, langsmith-0.3.11, asyncio-0.25.3
asyncio: mode=Mode.STRICT, asyncio_default_fixture_loop_scope=None
collected 66 items

tests/test_cache.py::TestCacheSetGet::test_basic_set_and_get PASSED
tests/test_cache.py::TestCacheSetGet::test_multiple_addresses PASSED
tests/test_cache.py::TestCacheSetGet::test_trailing_dot_normalised PASSED
tests/test_cache.py::TestCacheSetGet::test_miss_returns_none PASSED
tests/test_cache.py::TestCacheSetGet::test_empty_address_list_not_stored PASSED
tests/test_cache.py::TestCacheGetWithTTL::test_returns_addresses_and_positive_ttl PASSED
tests/test_cache.py::TestCacheGetWithTTL::test_miss_returns_none_and_zero PASSED
tests/test_cache.py::TestCacheGetWithTTL::test_remaining_ttl_less_than_original PASSED
tests/test_cache.py::TestCacheGetWithTTL::test_expired_entry_returns_none_and_zero PASSED
tests/test_cache.py::TestCacheTTL::test_expired_entry_returns_none PASSED
tests/test_cache.py::TestCacheTTL::test_ttl_clamped_to_min PASSED
tests/test_cache.py::TestCacheTTL::test_ttl_clamped_to_max PASSED
tests/test_cache.py::TestCacheInvalidate::test_invalidate_existing PASSED
tests/test_cache.py::TestCacheInvalidate::test_invalidate_missing PASSED
tests/test_cache.py::TestCacheHitCount::test_hit_count_increments PASSED

```

Figure 16: Complete pytest execution